Zip Code Group Project

2.0

Generated by Doxygen 1.9.8

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 Buffer Class Reference . . . 5

3.1.1 Detailed Description . . . 6

3.1.2 Constructor & Destructor Documentation . . . 6

3.1.2.1 Buffer() . . . 6

3.1.3 Member Function Documentation . . . 7

3.1.3.1 displayTable() . . . 7

3.1.3.2 findExtrema() . . . 7

3.1.3.3 getEasternmostZipCode() . . . 8

3.1.3.4 getLatitude() . . . 9

3.1.3.5 getLongitude() . . . 10

3.1.3.6 getNorthernmostZipCode() . . . 10

3.1.3.7 getPlaceName() . . . 11

3.1.3.8 getSouthernmostZipCode() . . . 11

3.1.3.9 getState() . . . 12

3.1.3.10 getWesternmostZipCode() . . . 12

3.1.3.11 getZipCode() . . . 13

3.1.3.12 readCSV() . . . 13

3.1.3.13 setLatitude() . . . 15

3.1.3.14 setLongitude() . . . 15

3.1.3.15 setPlaceName() . . . 16

3.1.3.16 setState() . . . 16

3.1.3.17 setZipCode() . . . 17

3.1.4 Member Data Documentation . . . 17

3.1.4.1 extremes . . . 17

3.1.4.2 latitude . . . 17

3.1.4.3 longitude . . . 18

3.1.4.4 placeName . . . 18

3.1.4.5 state . . . 18

3.1.4.6 zipCode . . . 18

3.2 Extremes Struct Reference . . . 19

3.2.1 Detailed Description . . . 19

3.2.2 Member Data Documentation . . . 19

3.2.2.1 east . . . 19

3.2.2.2 eastLongitude . . . 20
3.2.2.3 north . . . 20
3.2.2.4 northLatitude . . . 20
3.2.2.5 south . . . 20
3.2.2.6 southLatitude . . . 20
3.2.2.7 west . . . 21
3.2.2.8 westLongitude . . . 21
3.3 FieldInfo Struct Reference . . . 21
3.3.1 Detailed Description . . . 21
3.3.2 Member Data Documentation . . . 21
3.3.2.1 fieldName . . . 21
3.3.2.2 fieldType . . . 22
3.4 FileHeader Struct Reference . . . 22
3.4.1 Detailed Description . . . 22
3.4.2 Member Data Documentation . . . 22
3.4.2.1 fieldCount . . . 22
3.4.2.2 fields . . . 22
3.4.2.3 headerSize . . . 23
3.4.2.4 primaryKeyName . . . 23
3.4.2.5 recordCount . . . 23
3.4.2.6 sizeFieldBytes . . . 23
3.4.2.7 sizeincluded . . . 23
3.4.2.8 structureType . . . 23
3.4.2.9 version . . . 24
3.5 Index Class Reference . . . 24
3.5.1 Detailed Description . . . 24
3.5.2 Member Function Documentation . . . 25
3.5.2.1 build() . . . 25
3.5.2.2 find() . . . 25
3.5.3 Member Data Documentation . . . 26
3.5.3.1 entries . . . 26
3.6 LengthIndicatedBuffer Class Reference . . . 26
3.6.1 Detailed Description . . . 27
3.6.2 Constructor & Destructor Documentation . . . 27
3.6.2.1 LengthIndicatedBuffer() . . . 27
3.6.3 Member Function Documentation . . . 27
3.6.3.1 getFields() . . . 27
3.6.3.2 getRecord() . . . 28
3.6.3.3 pack() . . . 28

3.6.3.4 read() . . . 28
3.6.3.5 unpack() . . . 29
3.6.3.6 write() . . . 30
3.6.4 Member Data Documentation . . . 31
3.6.4.1 record . . . 31
3.7 Search Class Reference . . . 31
3.7.1 Detailed Description . . . 32
3.7.2 Member Function Documentation . . . 32
3.7.2.1 findZipCode() . . . 32
3.7.2.2 open() . . . 33
3.7.3 Member Data Documentation . . . 33
3.7.3.1 file . . . 33
3.7.3.2 index . . . 33

4 File Documentation **35**

4.1 CSVConversion.cpp File Reference . . . 35
4.1.1 Function Documentation . . . 36
4.1.1.1 analyzeCSV() . . . 36
4.1.1.2 convertCSVTodat() . . . 36
4.1.1.3 main() . . . 37
4.1.1.4 writeHeaderToFile() . . . 37
4.2 CSVConversion.cpp . . . 38
4.3 Index.cpp File Reference . . . 39
4.3.1 Detailed Description . . . 40
4.4 Index.cpp . . . 40
4.5 Index.h File Reference . . . 40
4.5.1 Detailed Description . . . 41
4.6 Index.h . . . 42
4.7 LengthIndicatedBuffer.cpp File Reference . . . 42
4.7.1 Detailed Description . . . 43
4.8 LengthIndicatedBuffer.cpp . . . 43
4.9 LengthIndicatedBuffer.h File Reference . . . 44
4.9.1 Detailed Description . . . 45
4.10 LengthIndicatedBuffer.h . . . 46
4.11 main.cpp File Reference . . . 46
4.11.1 Detailed Description . . . 47
4.11.2 Function Documentation . . . 47
4.11.2.1 main() . . . 47
4.12 main.cpp . . . 48

4.13 Search.cpp File Reference 49
4.13.1 Detailed Description 50
4.14 Search.cpp 50
4.15 Search.h File Reference 51
4.15.1 Detailed Description 52
4.16 Search.h 52
4.17 ZipCodeBuffer.cpp File Reference 52
4.17.1 Detailed Description 53
4.17.2 Function Documentation 53
4.17.2.1 parseDouble() 53
4.17.2.2 parseInteger() 54
4.17.2.3 splitCSV() 54
4.17.2.4 trim() 54
4.18 ZipCodeBuffer.cpp 55
4.19 ZipCodeBuffer.h File Reference 59
4.19.1 Detailed Description 60
4.20 ZipCodeBuffer.h 61

Index **63**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Buffer
: Reads records sequentially and accumulates geographic extrema 5

Extremes
: ZIP codes and comparison coordinates for one state 19

Fieldinfo 21

FileHeader 22

Index 24

LengthIndicatedBuffer
: Reads and writes one length indicated record at a time 26

Search 31

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

CSVConversion.cpp 35
Index.cpp
    Builds a ZIP code index without assuming CSV record order 39
Index.h
    Maps ZIP codes to record positions in a CSV file 40
LengthIndicatedBuffer.cpp
    Implements the length indicated buffer 42
LengthIndicatedBuffer.h
    Buffer for length indicated records 44
main.cpp
    Application entry point for the ZIP Code group project 46
Search.cpp
    Seeks directly to indexed ZIP records and unpacks their fields 49
Search.h
    Retrieves indexed CSV records into the existing Buffer class 51
ZipCodeBuffer.cpp
    Implements ZIP code CSV parsing and geographic extrema 52
ZipCodeBuffer.h
    Record buffer and per-state geographic extrema declarations 59

Chapter 3

Class Documentation

3.1 Buffer Class Reference

Reads records sequentially and accumulates geographic extrema.

`#include <ZipCodeBuffer.h>`

Public Member Functions

- Buffer ()

  Initializes an empty record and empty extrema map.
- void setState (string state)

  Sets the current record State.
- void setPlaceName (string placeName)

  Sets the current record PlaceName.
- void setLatitude (double latitude)

  Sets the current record Latitude.
- void setLongitude (double longitude)

  Sets the current record Longitude.
- void setZipCode (int zipCode)

  Sets the current record ZipCode.
- string getState ()

  Returns State.
- string getPlaceName ()

  Returns PlaceName.
- double getLatitude ()

  Returns Latitude.
- double getLongitude ()

  Returns Longitude.
- int getZipCode ()

  Returns ZipCode.
- bool readCSV (string fileName)

Reads the CSV and accumulates one record at a time.

- void findExtrema ()

  Updates state extrema using the current buffered record.
- int getEasternmostZipCode (string state)

  Returns EasternmostZipCode.
- int getWesternmostZipCode (string state)

  Returns WesternmostZipCode.
- int getNorthernmostZipCode (string state)

  Returns NorthernmostZipCode.
- int getSouthernmostZipCode (string state)

  Returns SouthernmostZipCode.
- void displayTable ()

  Prints the state extrema table to standard output.

Private Attributes

- string state

  Current record state.
- string placeName

  Current record placeName.
- double latitude

  Current record latitude.
- double longitude

  Current record longitude.
- int zipCode

  Current record zipCode.
- map< string, Extremes > extremes

3.1.1 Detailed Description

Reads records sequentially and accumulates geographic extrema.

Stores the current record and a state-keyed map. Equal coordinates select the smaller ZIP code, making results independent of record order.

Definition at line 29 of file ZipCodeBuffer.h.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 Buffer()

```
Buffer::Buffer ( )
```

Initializes an empty record and empty extrema map.

Initializes an empty ZIP code record.

Definition at line 82 of file ZipCodeBuffer.cpp.

```
00082                {
00083     state = "";
00084     placeName = "";
00085     latitude = 0.0;
00086     longitude = 0.0;
00087     zipCode = 0;
00088 }
```

References latitude, longitude, placeName, state, and zipCode.

3.1.3 Member Function Documentation

3.1.3.1 displayTable()

```
void Buffer::displayTable ( )
```

Prints the state extrema table to standard output.

Displays geographic extrema sorted by state.

Rows follow alphabetical state order. Columns are state, east, west, north, south. ZIP codes are padded to five digits.

Definition at line 325 of file ZipCodeBuffer.cpp.

```
00325                           {
00326
00327     cout « left « setfill(' ');
00328     cout « setw(7) « "State"
00329          « setw(14) « "Easternmost"
00330          « setw(14) « "Westernmost"
00331          « setw(14) « "Northernmost"
00332          « setw(14) « "Southernmost"
00333          « endl;
00334
00335     cout « string(63, '-') « endl;
00336
00337     for (const auto& entry : extremes) {
00338         const string& stateValue = entry.first;
00339
00340         cout « left « setfill(' ')
00341              « setw(7) « stateValue;
00342
00343         cout « right « setfill('0');
00344
00345         cout « setw(5)
00346              « getEasternmostZipCode(stateValue)
00347              « "         ";
00348
00349         cout « setw(5)
00350              « getWesternmostZipCode(stateValue)
00351              « "         ";
00352
00353         cout « setw(5)
00354              « getNorthernmostZipCode(stateValue)
00355              « "         ";
00356
00357         cout « setw(5)
00358              « getSouthernmostZipCode(stateValue)
00359              « endl;
00360     }
00361
00362     cout « setfill(' ') « left;
00363 }
```

References extremes, getEasternmostZipCode(), getNorthernmostZipCode(), getSouthernmostZipCode(), and getWesternmostZipCode().

Referenced by main().

3.1.3.2 findExtrema()

```
void Buffer::findExtrema ( )
```

Updates state extrema using the current buffered record.

Finds easternmost, westernmost, northernmost, and southernmost ZIP codes for each state.

East uses maximum longitude, west minimum longitude, north maximum latitude, and south minimum latitude. Ties use the smaller ZIP.

Coordinate ties select the smaller ZIP code.

Definition at line 240 of file ZipCodeBuffer.cpp.

```
00240                           {
00241
00242     if (extremes.count(state) == 0) {
00243         Extremes extreme;
00244
00245         extreme.east = zipCode;
00246         extreme.west = zipCode;
00247         extreme.north = zipCode;
00248         extreme.south = zipCode;
00249
00250         extreme.eastLongitude = longitude;
00251         extreme.westLongitude = longitude;
00252         extreme.northLatitude = latitude;
00253         extreme.southLatitude = latitude;
00254
00255         extremes[state] = extreme;
00256         return;
00257     }
00258
00259     Extremes& extreme = extremes[state];
00260
00261     // Easternmost: highest longitude.
00262     if (longitude > extreme.eastLongitude ||
00263         (longitude == extreme.eastLongitude &&
00264          zipCode < extreme.east)) {
00265         extreme.east = zipCode;
00266         extreme.eastLongitude = longitude;
00267     }
00268
00269     // Westernmost: lowest longitude.
00270     if (longitude < extreme.westLongitude ||
00271         (longitude == extreme.westLongitude &&
00272          zipCode < extreme.west)) {
00273         extreme.west = zipCode;
00274         extreme.westLongitude = longitude;
00275     }
00276
00277     // Northernmost: highest latitude.
00278     if (latitude > extreme.northLatitude ||
00279         (latitude == extreme.northLatitude &&
00280          zipCode < extreme.north)) {
00281         extreme.north = zipCode;
00282         extreme.northLatitude = latitude;
00283     }
00284
00285     // Southernmost: lowest latitude.
00286     if (latitude < extreme.southLatitude ||
00287         (latitude == extreme.southLatitude &&
00288          zipCode < extreme.south)) {
00289         extreme.south = zipCode;
00290         extreme.southLatitude = latitude;
00291     }
00292 }
```

References Extremes::east, Extremes::eastLongitude, extremes, latitude, longitude, Extremes::north, Extremes::northLatitude, Extremes::south, Extremes::southLatitude, state, Extremes::west, Extremes::westLongitude, and zipCode.

Referenced by readCSV().

3.1.3.3 getEasternmostZipCode()

```
int Buffer::getEasternmostZipCode (
            string state )
```

Returns EasternmostZipCode.

Returns the easternmost ZIP code.

Parameters

state	State abbreviation already present in the extrema map.

Precondition

The state has been accumulated by findExtrema().

Returns

ZIP code for the requested direction.

Note

An unknown state inserts a zero-initialized map entry.

Definition at line 297 of file ZipCodeBuffer.cpp.

```
00297                                                   {
00298     return extremes.at(state).east;
00299 }
```

References extremes, and state.

Referenced by displayTable().

3.1.3.4 getLatitude()

```
double Buffer::getLatitude ( )
```

Returns Latitude.

Returns

Stored value.

Definition at line 118 of file ZipCodeBuffer.cpp.

```
00118                                {
00119     return latitude;
00120 }
```

References latitude.

Referenced by Search::findZipCode().

3.1.3.5 getLongitude()

```
double Buffer::getLongitude ( )
```

Returns Longitude.

Returns

Stored value.

Definition at line 122 of file ZipCodeBuffer.cpp.

```
00122                               {
00123     return longitude;
00124 }
```

References longitude.

Referenced by Search::findZipCode().

3.1.3.6 getNorthernmostZipCode()

```
int Buffer::getNorthernmostZipCode (
            string state )
```

Returns NorthernmostZipCode.

Returns the northernmost ZIP code.

Parameters

state	State abbreviation already present in the extrema map.

Precondition

The state has been accumulated by findExtrema().

Returns

ZIP code for the requested direction.

Note

An unknown state inserts a zero-initialized map entry.

Definition at line 311 of file ZipCodeBuffer.cpp.

```
00311                                                   {
00312     return extremes.at(state).north;
00313 }
```

References extremes, and state.

Referenced by displayTable().

3.1.3.7 getPlaceName()

```
string Buffer::getPlaceName ( )
```

Returns PlaceName.

Returns

Stored value.

Definition at line 114 of file ZipCodeBuffer.cpp.

```
00114                                {
00115     return placeName;
00116 }
```

References placeName.

Referenced by Search::findZipCode().

3.1.3.8 getSouthernmostZipCode()

```
int Buffer::getSouthernmostZipCode (
             string state )
```

Returns SouthernmostZipCode.

Returns the southernmost ZIP code.

Parameters

state	State abbreviation already present in the extrema map.

Precondition

The state has been accumulated by findExtrema().

Returns

ZIP code for the requested direction.

Note

An unknown state inserts a zero-initialized map entry.

Definition at line 318 of file ZipCodeBuffer.cpp.

```
00318                                                {
00319     return extremes.at(state).south;
00320 }
```

References extremes, and state.

Referenced by displayTable().

3.1.3.9 getState()

```
string Buffer::getState ( )
```

Returns State.

Returns

Stored value.

Definition at line 110 of file ZipCodeBuffer.cpp.

```
00110                           {
00111     return state;
00112 }
```

References state.

Referenced by Search::findZipCode().

3.1.3.10 getWesternmostZipCode()

```
int Buffer::getWesternmostZipCode (
            string state )
```

Returns WesternmostZipCode.

Returns the westernmost ZIP code.

Parameters

state	State abbreviation already present in the extrema map.

Precondition

The state has been accumulated by findExtrema().

Returns

ZIP code for the requested direction.

Note

An unknown state inserts a zero-initialized map entry.

Definition at line 304 of file ZipCodeBuffer.cpp.

```
00304                                                   {
00305     return extremes.at(state).west;
00306 }
```

References extremes, and state.

Referenced by displayTable().

3.1.3.11 getZipCode()

```
int Buffer::getZipCode ( )
```

Returns ZipCode.

Returns

Stored value.

Definition at line 126 of file ZipCodeBuffer.cpp.

```
00126                       {
00127     return zipCode;
00128 }
```

References zipCode.

Referenced by Search::findZipCode().

3.1.3.12 readCSV()

```
bool Buffer::readCSV (
            string fileName )
```

Reads the CSV and accumulates one record at a time.

Reads ZIP code records from a CSV file.

Parameters

fileName	Path to a CSV with a header and columns ZIP, place, state, county, latitude, longitude.

Returns

false if opening fails; true after processing the file.

Skips records with empty ZIP, state, latitude, or longitude. Existing extrema remain when called again. Comma splitting does not handle quoted commas.

Exceptions

std::invalid_argument	If a numeric field cannot be converted.
std::out_of_range	If a numeric field is outside the conversion range.

Parameters

fileName	CSV file path.

Returns

true if opened and read; false otherwise.

Invalid or incomplete records are skipped. Existing extrema are preserved across calls.

Definition at line 138 of file ZipCodeBuffer.cpp.

```
00138                                       {
00139     ifstream file(fileName);
00140
00141     if (!file.is_open()) {
00142         cerr « "Error: Cannot open " « fileName « endl;
00143         return false;
00144     }
00145
00146     string line;
00147     size_t lineNumber = 0;
00148     size_t processed = 0;
00149     size_t skipped = 0;
00150
00151     // Skip CSV header.
00152     if (!getline(file, line)) {
00153         cerr « "Error: Empty CSV: " « fileName « endl;
00154         return false;
00155     }
00156
00157     ++lineNumber;
00158
00159     while (getline(file, line)) {
00160         ++lineNumber;
00161
00162         vector<string> fields = splitCSV(line);
00163
00164         // Expected columns:
00165         // ZIP, Place, State, County, Latitude, Longitude
00166         if (fields.size() < 6) {
00167             ++skipped;
00168             continue;
00169         }
00170
00171         const string& zip = fields[0];
00172         const string& place = fields[1];
00173         const string& stateValue = fields[2];
00174         const string& lat = fields[4];
00175         const string& lon = fields[5];
00176
00177         if (zip.empty() || stateValue.empty() ||
00178             lat.empty() || lon.empty()) {
00179             ++skipped;
00180             continue;
00181         }
00182
00183         try {
00184             int parsedZip = parseInteger(zip);
00185             double parsedLat = parseDouble(lat);
00186             double parsedLon = parseDouble(lon);
00187
00188             if (parsedZip < 0 || parsedZip > 99999 ||
00189                 parsedLat < -90 || parsedLat > 90 ||
00190                 parsedLon < -180 || parsedLon > 180) {
00191                 ++skipped;
00192                 continue;
00193             }
00194
00195             setZipCode(parsedZip);
00196             setPlaceName(place);
00197             setState(stateValue);
00198             setLatitude(parsedLat);
00199             setLongitude(parsedLon);
00200
00201             findExtrema();
00202             ++processed;
00203
00204         } catch (const exception& e) {
00205             ++skipped;
00206
00207             // Limit diagnostic output for large datasets.
00208             if (skipped <= 10) {
```

```
00209                 cerr « "Skipping invalid record at line "
00210                      « lineNumber « " in " « fileName
00211                      « ": " « e.what() « endl;
00212             }
00213         }
00214     }
00215
00216     if (file.bad()) {
00217         cerr « "Error reading file: " « fileName « endl;
00218         return false;
00219     }
00220
00221     cout « "File: " « fileName « endl;
00222     cout « "Processed records: " « processed « endl;
00223     cout « "Skipped records: " « skipped « endl;
00224
00225     if (processed == 0) {
00226         cerr « "Error: No valid records in "
00227              « fileName « endl;
00228         return false;
00229     }
00230
00231     return true;
00232 }
```

References findExtrema(), parseDouble(), parseInteger(), setLatitude(), setLongitude(), setPlaceName(), setState(), setZipCode(), and splitCSV().

Referenced by main().

3.1.3.13 setLatitude()

```
void Buffer::setLatitude (
             double latitude )
```

Sets the current record Latitude.

Parameters

latitude	Value to store in the record buffer.

Definition at line 98 of file ZipCodeBuffer.cpp.

```
00098                                          {
00099     this->latitude = latitude;
00100 }
```

References latitude.

Referenced by Search::findZipCode(), readCSV(), and LengthIndicatedBuffer::unpack().

3.1.3.14 setLongitude()

```
void Buffer::setLongitude (
             double longitude )
```

Sets the current record Longitude.

Parameters

longitude	Value to store in the record buffer.

Definition at line 102 of file ZipCodeBuffer.cpp.

```
00102                                           {
00103     this->longitude = longitude;
00104 }
```

References longitude.

Referenced by Search::findZipCode(), readCSV(), and LengthIndicatedBuffer::unpack().

3.1.3.15 setPlaceName()

```
void Buffer::setPlaceName (
             string placeName )
```

Sets the current record PlaceName.

Parameters

placeName	Value to store in the record buffer.

Definition at line 94 of file ZipCodeBuffer.cpp.

```
00094                                           {
00095     this->placeName = placeName;
00096 }
```

References placeName.

Referenced by Search::findZipCode(), readCSV(), and LengthIndicatedBuffer::unpack().

3.1.3.16 setState()

```
void Buffer::setState (
             string state )
```

Sets the current record State.

Parameters

state	Value to store in the record buffer.

Definition at line 90 of file ZipCodeBuffer.cpp.

```
00090                                       {
00091     this->state = state;
00092 }
```

References state.

Referenced by Search::findZipCode(), readCSV(), and LengthIndicatedBuffer::unpack().

3.1.3.17 setZipCode()

```
void Buffer::setZipCode (
            int zipCode )
```

Sets the current record ZipCode.

Parameters

zipCode	Value to store in the record buffer.

Definition at line 106 of file ZipCodeBuffer.cpp.

```
00106                                   {
00107     this->zipCode = zipCode;
00108 }
```

References zipCode.

Referenced by Search::findZipCode(), readCSV(), and LengthIndicatedBuffer::unpack().

3.1.4 Member Data Documentation

3.1.4.1 extremes

```
map<string, Extremes> Buffer::extremes  [private]
```

Definition at line 147 of file ZipCodeBuffer.h.

Referenced by displayTable(), findExtrema(), getEasternmostZipCode(), getNorthernmostZipCode(), getSouthernmostZipCode(), and getWesternmostZipCode().

3.1.4.2 latitude

```
double Buffer::latitude  [private]
```

Current record latitude.

Definition at line 143 of file ZipCodeBuffer.h.

Referenced by Buffer(), findExtrema(), getLatitude(), and setLatitude().

3.1.4.3 longitude

```
double Buffer::longitude  [private]
```

Current record longitude.

Definition at line 144 of file ZipCodeBuffer.h.

Referenced by Buffer(), findExtrema(), getLongitude(), and setLongitude().

3.1.4.4 placeName

```
string Buffer::placeName  [private]
```

Current record placeName.

Definition at line 142 of file ZipCodeBuffer.h.

Referenced by Buffer(), getPlaceName(), and setPlaceName().

3.1.4.5 state

```
string Buffer::state  [private]
```

Current record state.

Definition at line 141 of file ZipCodeBuffer.h.

Referenced by Buffer(), findExtrema(), getEasternmostZipCode(), getNorthernmostZipCode(), getSouthernmostZipCode(), getState(), getWesternmostZipCode(), and setState().

3.1.4.6 zipCode

```
int Buffer::zipCode  [private]
```

Current record zipCode.

Definition at line 145 of file ZipCodeBuffer.h.

Referenced by Buffer(), findExtrema(), getZipCode(), and setZipCode().

The documentation for this class was generated from the following files:

- ZipCodeBuffer.h
- ZipCodeBuffer.cpp

3.2 Extremes Struct Reference

ZIP codes and comparison coordinates for one state.

`#include <ZipCodeBuffer.h>`

Public Attributes

- int east
  Stored east value.
- int west
  Stored west value.
- int north
  Stored north value.
- int south
  Stored south value.
- double eastLongitude
  Stored eastLongitude value.
- double westLongitude
  Stored westLongitude value.
- double northLatitude
  Stored northLatitude value.
- double southLatitude
  Stored southLatitude value.

3.2.1 Detailed Description

ZIP codes and comparison coordinates for one state.

Definition at line 13 of file ZipCodeBuffer.h.

3.2.2 Member Data Documentation

3.2.2.1 east

`int Extremes::east`

Stored east value.

Definition at line 14 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.2 eastLongitude

```
double Extremes::eastLongitude
```

Stored eastLongitude value.

Definition at line 19 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.3 north

```
int Extremes::north
```

Stored north value.

Definition at line 16 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.4 northLatitude

```
double Extremes::northLatitude
```

Stored northLatitude value.

Definition at line 21 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.5 south

```
int Extremes::south
```

Stored south value.

Definition at line 17 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.6 southLatitude

```
double Extremes::southLatitude
```

Stored southLatitude value.

Definition at line 22 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.7 west

```
int Extremes::west
```

Stored west value.

Definition at line 15 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

3.2.2.8 westLongitude

```
double Extremes::westLongitude
```

Stored westLongitude value.

Definition at line 20 of file ZipCodeBuffer.h.

Referenced by Buffer::findExtrema().

The documentation for this struct was generated from the following file:

- ZipCodeBuffer.h

3.3 Fieldinfo Struct Reference

Public Attributes

- std::string fieldName
- int fieldType

3.3.1 Detailed Description

Definition at line 22 of file CSVConversion.cpp.

3.3.2 Member Data Documentation

3.3.2.1 fieldName

```
std::string Fieldinfo::fieldName
```

Definition at line 23 of file CSVConversion.cpp.

3.3.2.2 fieldType

```
int Fieldinfo::fieldType
```

Definition at line 24 of file CSVConversion.cpp.

The documentation for this struct was generated from the following file:

- CSVConversion.cpp

3.4 FileHeader Struct Reference

Public Attributes

- std::string structureType = "Length indicated"
- int version = 1
- int headerSize = 0
- int sizeFieldBytes = 4
- bool sizeincluded = false
- std::string primaryKeyName = "Zip Code"
- int recordCount = 0
- int fieldCount = 0
- vector< Fieldinfo > fields

3.4.1 Detailed Description

Definition at line 8 of file CSVConversion.cpp.

3.4.2 Member Data Documentation

3.4.2.1 fieldCount

```
int FileHeader::fieldCount = 0
```

Definition at line 18 of file CSVConversion.cpp.

Referenced by convertCSVTodat(), and writeHeaderToFile().

3.4.2.2 fields

```
vector<Fieldinfo> FileHeader::fields
```

Definition at line 20 of file CSVConversion.cpp.

3.4.2.3 headerSize

```
int FileHeader::headerSize = 0
```

Definition at line 11 of file CSVConversion.cpp.

Referenced by writeHeaderToFile().

3.4.2.4 primaryKeyName

```
std::string FileHeader::primaryKeyName = "Zip Code"
```

Definition at line 15 of file CSVConversion.cpp.

Referenced by writeHeaderToFile().

3.4.2.5 recordCount

```
int FileHeader::recordCount = 0
```

Definition at line 17 of file CSVConversion.cpp.

Referenced by convertCSVTodat(), and writeHeaderToFile().

3.4.2.6 sizeFieldBytes

```
int FileHeader::sizeFieldBytes = 4
```

Definition at line 12 of file CSVConversion.cpp.

Referenced by convertCSVTodat(), and writeHeaderToFile().

3.4.2.7 sizeincluded

```
bool FileHeader::sizeincluded = false
```

Definition at line 13 of file CSVConversion.cpp.

Referenced by writeHeaderToFile().

3.4.2.8 structureType

```
std::string FileHeader::structureType = "Length indicated"
```

Definition at line 9 of file CSVConversion.cpp.

Referenced by writeHeaderToFile().

3.4.2.9 version

```
int FileHeader::version = 1
```

Definition at line 10 of file CSVConversion.cpp.

Referenced by writeHeaderToFile().

The documentation for this struct was generated from the following file:

- CSVConversion.cpp

3.5 Index Class Reference

```
#include <Index.h>
```

Public Member Functions

- bool build (const std::string &fileName)

  Builds an index for a CSV with ZIP code in its first column.
- bool find (int zipCode, std::streamoff &position) const

  Finds a ZIP's byte offset without changing position on failure.

Private Attributes

- std::map< int, std::streamoff > entries

3.5.1 Detailed Description

Definition at line 11 of file Index.h.

3.5.2 Member Function Documentation

3.5.2.1 build()

```
bool Index::build (
            const std::string & fileName )
```

Builds an index for a CSV with ZIP code in its first column.

Returns

false on an open or read error; the index is then empty.

Skips the header and invalid ZIP fields. For duplicate ZIP codes, keeps the first record. Rebuild after changing the CSV file.

Definition at line 8 of file Index.cpp.

```
00008                                           {
00009     entries.clear();
00010     // Binary mode keeps seek positions correct for Windows CRLF files.
00011     std::ifstream file(fileName, std::ios::binary);
00012     if (!file.is_open()) {
00013         return false;
00014     }
00015
00016     std::string line;
00017     std::getline(file, line); // Skip the column names.
00018     while (file) {
00019         std::streamoff position = file.tellg();
00020         if (!std::getline(file, line)) {
00021             break;
00022         }
00023         std::size_t comma = line.find(',');
00024         if (comma == std::string::npos) {
00025             continue;
00026         }
00027         std::string zip = line.substr(0, comma);
00028         if (zip.empty() || zip.size() > 5 ||
00029             zip.find_first_not_of("0123456789") != std::string::npos) {
00030             continue;
00031         }
00032         entries.insert(std::make_pair(std::stoi(zip), position));
00033     }
00034     if (file.bad() || (file.fail() && !file.eof())) {
00035         entries.clear();
00036         return false;
00037     }
00038     return true;
00039 }
```

References entries.

Referenced by Search::open().

3.5.2.2 find()

```
bool Index::find (
            int zipCode,
            std::streamoff & position ) const
```

Finds a ZIP's byte offset without changing position on failure.

Definition at line 41 of file Index.cpp.

```
00041                                                   {
00042     auto entry = entries.find(zipCode);
00043     if (entry == entries.end()) {
00044         return false;
00045     }
00046     position = entry->second;
00047     return true;
00048 }
```

References entries.

Referenced by Search::findZipCode().

3.5.3 Member Data Documentation

3.5.3.1 entries

```
std::map<int, std::streamoff> Index::entries  [private]
```

Definition at line 24 of file Index.h.

Referenced by build(), and find().

The documentation for this class was generated from the following files:

- Index.h
- Index.cpp

3.6 LengthIndicatedBuffer Class Reference

Reads and writes one length indicated record at a time.

```
#include <LengthIndicatedBuffer.h>
```

Public Member Functions

- LengthIndicatedBuffer ()
  Constructor.
- bool read (istream &file)
  Reads the next record.
- bool unpack (Buffer &buffer, vector< string > fieldNames)
  Unpacks the record into a Buffer.
- void pack (string line)
  Stores a record.
- void write (ostream &file)
  Writes the record with its length.
- string getRecord ()
  Returns the record.
- vector< string > getFields ()
  Returns the record's fields.

Private Attributes

- string record

3.6.1 Detailed Description

Reads and writes one length indicated record at a time.

Definition at line 13 of file LengthIndicatedBuffer.h.

3.6.2 Constructor & Destructor Documentation

3.6.2.1 LengthIndicatedBuffer()

```
LengthIndicatedBuffer::LengthIndicatedBuffer ( )
```

Constructor.

Definition at line 9 of file LengthIndicatedBuffer.cpp.

```
00009                                               {
00010     record = "";
00011 }
```

References record.

3.6.3 Member Function Documentation

3.6.3.1 getFields()

```
vector< string > LengthIndicatedBuffer::getFields ( )
```

Returns the record's fields.

Definition at line 103 of file LengthIndicatedBuffer.cpp.

```
00103                                                {
00104     vector<string> fields;
00105     stringstream ss(record);
00106     string field;
00107
00108     while(getline(ss, field, ','){
00109         fields.push_back(field);
00110     }
00111
00112     return fields;
00113 }
```

References record.

Referenced by unpack().

3.6.3.2 getRecord()

```
string LengthIndicatedBuffer::getRecord ( )
```

Returns the record.

Definition at line 99 of file LengthIndicatedBuffer.cpp.

```
00099                                       {
00100     return record;
00101 }
```

References record.

3.6.3.3 pack()

```
void LengthIndicatedBuffer::pack (
              string line )
```

Stores a record.

Parameters

line	Comma separated fields.

Definition at line 86 of file LengthIndicatedBuffer.cpp.

```
00086                                              {
00087     // take off the \r so it isn't counted in the length
00088     if(line != "" && line.back() == '\r'){
00089         line.pop_back();
00090     }
00091     record = line;
00092 }
```

References record.

Referenced by Search::findZipCode().

3.6.3.4 read()

```
bool LengthIndicatedBuffer::read (
              istream & file )
```

Reads the next record.

Parameters

file	Input file.

Returns

false at end of file or bad record.

Definition at line 13 of file LengthIndicatedBuffer.cpp.

```
00013                                                 {
00014     // The first 4 characters are the length
00015     char lengthText[5] = "";
00016     if(!file.read(lengthText, 4)){
00017         return false;
00018     }
00019
00020     int length = atoi(lengthText);
00021     if(length <= 0){
00022         cout « "Error: bad record length " « lengthText « endl;
00023         return false;
00024     }
00025
00026     // Read that many characters for the record
00027     record = string(length, ' ');
00028     if(!file.read(&record[0], length)){
00029         cout « "Error: record is cut off" « endl;
00030         return false;
00031     }
00032
00033     // skip the newline at the end (\r\n on Windows)
00034     if(file.peek() == '\r'){
00035         file.get();
00036     }
00037     if(file.peek() == '\n'){
00038         file.get();
00039     }
00040
00041     return true;
00042 }
```

References record.

3.6.3.5 unpack()

```
bool LengthIndicatedBuffer::unpack (
            Buffer & buffer,
            vector< string > fieldNames )
```

Unpacks the record into a Buffer.

Parameters

buffer	Buffer to fill.
fieldNames	Field names in order.

Returns

false if the record is missing data.

Definition at line 44 of file LengthIndicatedBuffer.cpp.

```
00044                                                                                 {
00045     vector<string> fields = getFields();
00046
00047     string zip;
00048     string place;
00049     string state;
```

```
00050     string lat;
00051     string lon;
00052
00053     // Use the field names so it still works if the columns are in a different order
00054     for(int i = 0; i < fields.size() && i < fieldNames.size(); i++){
00055         if(fieldNames[i] == "Zip Code"){
00056             zip = fields[i];
00057         }
00058         else if(fieldNames[i] == "Place Name"){
00059             place = fields[i];
00060         }
00061         else if(fieldNames[i] == "State"){
00062             state = fields[i];
00063         }
00064         else if(fieldNames[i] == "Lat"){
00065             lat = fields[i];
00066         }
00067         else if(fieldNames[i] == "Long"){
00068             lon = fields[i];
00069         }
00070     }
00071
00072     // skip any record that is missing data
00073     if(zip == "" || state == "" || lat == "" || lon == ""){
00074         return false;
00075     }
00076
00077     buffer.setZipCode(stoi(zip));
00078     buffer.setPlaceName(place);
00079     buffer.setState(state);
00080     buffer.setLatitude(stod(lat));
00081     buffer.setLongitude(stod(lon));
00082
00083     return true;
00084 }
```

References getFields(), Buffer::setLatitude(), Buffer::setLongitude(), Buffer::setPlaceName(), Buffer::setState(), and Buffer::setZipCode().

Referenced by Search::findZipCode().

3.6.3.6 write()

```
void LengthIndicatedBuffer::write (
            ostream & file )
```

Writes the record with its length.

Parameters

file	Output file.

Definition at line 94 of file LengthIndicatedBuffer.cpp.

```
00094                                                {
00095     // setw and setfill('0') make the length 4 digits, like 0042
00096     file « setw(4) « setfill('0') « record.length() « record « endl;
00097 }
```

References record.

3.6.4 Member Data Documentation

3.6.4.1 record

```
string LengthIndicatedBuffer::record  [private]
```

Definition at line 49 of file LengthIndicatedBuffer.h.

Referenced by getFields(), getRecord(), LengthIndicatedBuffer(), pack(), read(), and write().

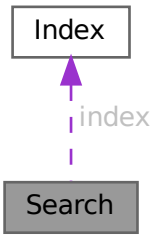
The documentation for this class was generated from the following files:

- LengthIndicatedBuffer.h
- LengthIndicatedBuffer.cpp

3.7 Search Class Reference

```
#include <Search.h>
```

Collaboration diagram for Search:



Public Member Functions

- bool open (const std::string &fileName)

  Opens and indexes a CSV in the existing six-column format.
- bool findZipCode (int zipCode, Buffer &buffer)

  Loads a ZIP record into buffer; leading zeros are optional.

Private Attributes

- Index index
- std::ifstream file

3.7.1 Detailed Description

Definition at line 12 of file Search.h.

3.7.2 Member Function Documentation

3.7.2.1 findZipCode()

```
bool Search::findZipCode (
            int zipCode,
            Buffer & buffer )
```

Loads a ZIP record into buffer; leading zeros are optional.

Returns

false if unopened, absent, unreadable, or malformed.

Leaves buffer unchanged on failure. Uses the existing comma splitting convention, which does not support quoted commas.

Definition at line 21 of file Search.cpp.

```
00021                                                   {
00022     std::streamoff position;
00023     if (!file.is_open() || !index.find(zipCode, position)) {
00024         return false;
00025     }
00026     file.clear();
00027     file.seekg(position);
00028     std::string line;
00029     if (!std::getline(file, line)) {
00030         return false;
00031     }
00032
00033     LengthIndicatedBuffer record;
00034     record.pack(line);
00035     Buffer result;
00036     const std::vector<std::string> fieldNames = {
00037         "Zip Code", "Place Name", "State", "County", "Lat", "Long"
00038     };
00039     try {
00040         if (!record.unpack(result, fieldNames) || result.getZipCode() != zipCode) {
00041             return false;
00042         }
00043     } catch (const std::invalid_argument&) {
00044         return false;
00045     } catch (const std::out_of_range&) {
00046         return false;
00047     }
00048
00049     // Copy only record fields; retain the caller's accumulated extrema.
00050     buffer.setZipCode(result.getZipCode());
00051     buffer.setPlaceName(result.getPlaceName());
00052     buffer.setState(result.getState());
00053     buffer.setLatitude(result.getLatitude());
00054     buffer.setLongitude(result.getLongitude());
00055     return true;
00056 }
```

References file, Index::find(), Buffer::getLatitude(), Buffer::getLongitude(), Buffer::getPlaceName(), Buffer::getState(), Buffer::getZipCode(), index, LengthIndicatedBuffer::pack(), Buffer::setLatitude(), Buffer::setLongitude(), Buffer::setPlaceName(), Buffer::setState(), Buffer::setZipCode(), and LengthIndicatedBuffer::unpack().

3.7.2.2 open()

```
bool Search::open (
            const std::string & fileName )
```

Opens and indexes a CSV in the existing six-column format.

Supports both ordered and randomized CSV files. The file must remain unchanged until the next open(). Does not read DAT files.

Returns

false if the file cannot be opened or indexed.

Definition at line 9 of file Search.cpp.

```
00009                                                {
00010     if (file.is_open()) {
00011         file.close();
00012     }
00013     file.clear();
00014     if (!index.build(fileName)) {
00015         return false;
00016     }
00017     file.open(fileName, std::ios::binary);
00018     return file.is_open();
00019 }
```

References Index::build(), file, and index.

3.7.3 Member Data Documentation

3.7.3.1 file

```
std::ifstream Search::file  [private]
```

Definition at line 30 of file Search.h.

Referenced by findZipCode(), and open().

3.7.3.2 index

```
Index Search::index  [private]
```

Definition at line 29 of file Search.h.

Referenced by findZipCode(), and open().

The documentation for this class was generated from the following files:

- Search.h
- Search.cpp

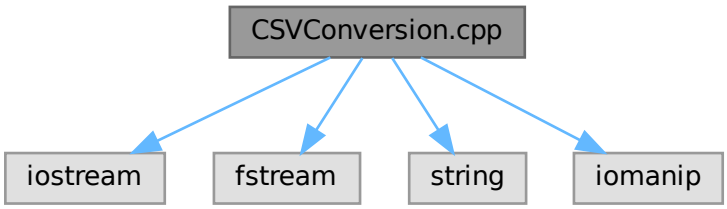
Chapter 4

File Documentation

4.1 CSVConversion.cpp File Reference

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>
```

Include dependency graph for CSVConversion.cpp:



Classes

- struct FileHeader
- struct Fieldinfo

Functions

- void writeHeaderToFile (ofstream &output, const FileHeader &fileHeader)
- void analyzeCSV (const string &csvFilePath, int &recordCount, int &fieldCount)
- string convertCSVTodat (const string &csvFilePath, const string &outputFilePath)
- int main ()

4.1.1 Function Documentation

4.1.1.1 analyzeCSV()

```
void analyzeCSV (
            const string & csvFilePath,
            int & recordCount,
            int & fieldCount )
```

Definition at line 39 of file CSVConversion.cpp.

```
00039                                                                                {
00040     ifstream csv(csvFilePath);
00041     if (!csv.is_open()) {
00042         cerr « "Error opening input file: " « csvFilePath « endl;
00043         return;
00044     }
00045
00046     string line;
00047     recordCount = 0;
00048     fieldCount = 0;
00049
00050     while (getline(csv, line)) {
00051         recordCount++;
00052         if (recordCount == 1) {
00053             // Count the number of fields in the first line
00054             size_t pos = 0;
00055             while ((pos = line.find(',', pos)) != string::npos) {
00056                 fieldCount++;
00057                 pos++; // Move past the comma
00058             }
00059             fieldCount++; // Add one for the last field
00060         }
00061     }
00062
00063     csv.close();
00064 }
```

Referenced by convertCSVTodat().

4.1.1.2 convertCSVTodat()

```
string convertCSVTodat (
            const string & csvFilePath,
            const string & outputFilePath )
```

Definition at line 65 of file CSVConversion.cpp.

```
00065                                                                                {
00066     FileHeader header;
00067
00068     analyzeCSV(csvFilePath, header.recordCount, header.fieldCount);
00069
00070     ifstream csv(csvFilePath);
00071     ofstream output(outputFilePath);
00072
00073     if (!csv.is_open()) {
00074         cerr « "Error opening input file: " « csvFilePath « endl;
00075         return "Error opening input file.";
00076     }
00077
00078     if (!output.is_open()) {
00079         cerr « "failed to create DAT file: " « outputFilePath « endl;
00080         return "failed to create DAT file.";
00081     }
00082
00083     writeHeaderToFile(output, header);
00084     string line;
00085
```

```
00086     while (getline(csv, line)) {
00087         int recordSize = line.length();
00088 
00089         output « setw(header.sizeFieldBytes) « setfill('0') « recordSize « line « endl; // Write the line
      to the output file
00090     }
00091 
00092     csv.close();
00093     output.close();
00094 
00095     return "Conversion completed successfully.";
00096 }
```

References analyzeCSV(), FileHeader::fieldCount, FileHeader::recordCount, FileHeader::sizeFieldBytes, and writeHeaderToFile().

Referenced by main().

4.1.1.3 main()

```
int main ( )
```

Definition at line 98 of file CSVConversion.cpp.

```
00098           {
00099     string csvFilePath1 = "us_postal_codes.csv"; // Path to the input CSV file
00100     string outputFilePath1 = "output.dat"; // Path to the output DAT file
00101     string csvFilePath2 = "us_postal_codes_ROWS_RANDOMIZED.csv"; // Path to the second input CSV file
00102     string outputFilePath2 = "output2.dat"; // Path to the second output DAT file
00103 
00104     string result1 = convertCSVTodat(csvFilePath1, outputFilePath1);
00105     cout « result1 « endl;
00106 
00107     string result2 = convertCSVTodat(csvFilePath2, outputFilePath2);
00108     cout « result2 « endl;
00109 
00110     return 0;
00111 }
```

References convertCSVTodat().

4.1.1.4 writeHeaderToFile()

```
void writeHeaderToFile (
            ofstream & output,
            const FileHeader & fileHeader )
```

Definition at line 26 of file CSVConversion.cpp.

```
00026                                                                   {
00027     output « "HEADER\n";
00028     output « "Structure Type: " « fileHeader.structureType « "\n";
00029     output « "Version: " « fileHeader.version « "\n";
00030     output « "Header Size: " « fileHeader.headerSize « "\n";
00031     output « "Size Field Bytes: " « fileHeader.sizeFieldBytes « "\n";
00032     output « "Size Included: " « (fileHeader.sizeincluded ? "true" : "false") « "\n";
00033     output « "Primary Key Name: " « fileHeader.primaryKeyName « "\n";
00034     output « "Record Count: " « fileHeader.recordCount « "\n";
00035     output « "Field Count: " « fileHeader.fieldCount « "\n";
00036     output « "END HEADER\n";
00037 }
```

References FileHeader::fieldCount, FileHeader::headerSize, FileHeader::primaryKeyName, FileHeader::recordCount, FileHeader::sizeFieldBytes, FileHeader::sizeincluded, FileHeader::structureType, and FileHeader::version.

Referenced by convertCSVTodat().

4.2 CSVConversion.cpp

Go to the documentation of this file.

```
00001 #include <iostream>
00002 #include <fstream>
00003 #include <string>
00004 #include <iomanip>
00005 using namespace std;
00006
00007
00008 struct FileHeader {
00009     std::string structureType= "Length indicated"; // 20 bytes for the structure type
00010     int version= 1;       // 4 bytes for the version
00011     int headerSize= 0;    // 4 bytes for the header size
00012     int sizeFieldBytes= 4; // 4 bytes for the size of each field
00013     bool sizeincluded= false; // 1 byte for the size included flag
00014
00015     std::string primaryKeyName= "Zip Code"; // 50 bytes for the primary key name
00016
00017     int recordCount= 0;   // 4 bytes for the number of records
00018     int fieldCount= 0;    // 4 bytes for the number of fields
00019
00020     vector<Fieldinfo> fields; // Vector to hold field information
00021 };
00022 struct Fieldinfo {
00023     std::string fieldName; // 50 bytes for the field name
00024     int fieldType;         // 4 bytes for the type of the field
00025 };
00026 void writeHeaderToFile(ofstream& output, const FileHeader& fileHeader) {
00027     output « "HEADER\n";
00028     output « "Structure Type: " « fileHeader.structureType « "\n";
00029     output « "Version: " « fileHeader.version « "\n";
00030     output « "Header Size: " « fileHeader.headerSize « "\n";
00031     output « "Size Field Bytes: " « fileHeader.sizeFieldBytes « "\n";
00032     output « "Size Included: " « (fileHeader.sizeincluded ? "true" : "false") « "\n";
00033     output « "Primary Key Name: " « fileHeader.primaryKeyName « "\n";
00034     output « "Record Count: " « fileHeader.recordCount « "\n";
00035     output « "Field Count: " « fileHeader.fieldCount « "\n";
00036     output « "END HEADER\n";
00037 }
00038
00039 void analyzeCSV(const string& csvFilePath, int& recordCount, int& fieldCount) {
00040     ifstream csv(csvFilePath);
00041     if (!csv.is_open()) {
00042         cerr « "Error opening input file: " « csvFilePath « endl;
00043         return;
00044     }
00045
00046     string line;
00047     recordCount = 0;
00048     fieldCount = 0;
00049
00050     while (getline(csv, line)) {
00051         recordCount++;
00052         if (recordCount == 1) {
00053             // Count the number of fields in the first line
00054             size_t pos = 0;
00055             while ((pos = line.find(',', pos)) != string::npos) {
00056                 fieldCount++;
00057                 pos++; // Move past the comma
00058             }
00059             fieldCount++; // Add one for the last field
00060         }
00061     }
00062
00063     csv.close();
00064 }
00065 string convertCSVTodat(const string& csvFilePath, const string& outputFilePath) {
00066     FileHeader header;
00067
00068     analyzeCSV(csvFilePath, header.recordCount, header.fieldCount);
00069
00070     ifstream csv(csvFilePath);
00071     ofstream output(outputFilePath);
00072
00073     if (!csv.is_open()) {
00074         cerr « "Error opening input file: " « csvFilePath « endl;
00075         return "Error opening input file.";
00076     }
```

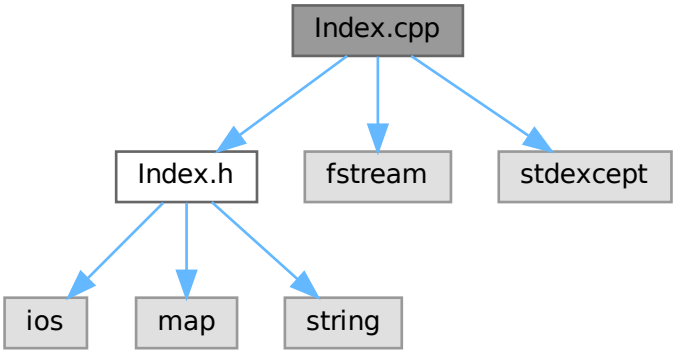
```
00077
00078     if (!output.is_open()) {
00079         cerr « "failed to create DAT file: " « outputFilePath « endl;
00080         return "failed to create DAT file.";
00081     }
00082
00083     writeHeaderToFile(output, header);
00084     string line;
00085
00086     while (getline(csv, line)) {
00087         int recordSize = line.length();
00088
00089         output « setw(header.sizeFieldBytes) « setfill('0') « recordSize « line « endl; // Write the line
      to the output file
00090     }
00091
00092     csv.close();
00093     output.close();
00094
00095     return "Conversion completed successfully.";
00096 }
00097
00098 int main() {
00099     string csvFilePath1 = "us_postal_codes.csv"; // Path to the input CSV file
00100     string outputFilePath1 = "output.dat"; // Path to the output DAT file
00101     string csvFilePath2 = "us_postal_codes_ROWS_RANDOMIZED.csv"; // Path to the second input CSV file
00102     string outputFilePath2 = "output2.dat"; // Path to the second output DAT file
00103
00104     string result1 = convertCSVTodat(csvFilePath1, outputFilePath1);
00105     cout « result1 « endl;
00106
00107     string result2 = convertCSVTodat(csvFilePath2, outputFilePath2);
00108     cout « result2 « endl;
00109
00110     return 0;
00111 }
```

4.3 Index.cpp File Reference

Builds a ZIP code index without assuming CSV record order.

```
#include "Index.h"
#include <fstream>
#include <stdexcept>
```

Include dependency graph for Index.cpp:



4.3.1 Detailed Description

Builds a ZIP code index without assuming CSV record order.

Definition in file Index.cpp.

4.4 Index.cpp

Go to the documentation of this file.

```
00001 /** @file Index.cpp
00002  * @brief Builds a ZIP code index without assuming CSV record order.
00003  */
00004 #include "Index.h"
00005 #include <fstream>
00006 #include <stdexcept>
00007
00008 bool Index::build(const std::string& fileName) {
00009     entries.clear();
00010     // Binary mode keeps seek positions correct for Windows CRLF files.
00011     std::ifstream file(fileName, std::ios::binary);
00012     if (!file.is_open()) {
00013         return false;
00014     }
00015
00016     std::string line;
00017     std::getline(file, line); // Skip the column names.
00018     while (file) {
00019         std::streamoff position = file.tellg();
00020         if (!std::getline(file, line)) {
00021             break;
00022         }
00023         std::size_t comma = line.find(',');
00024         if (comma == std::string::npos) {
00025             continue;
00026         }
00027         std::string zip = line.substr(0, comma);
00028         if (zip.empty() || zip.size() > 5 ||
00029             zip.find_first_not_of("0123456789") != std::string::npos) {
00030             continue;
00031         }
00032         entries.insert(std::make_pair(std::stoi(zip), position));
00033     }
00034     if (file.bad() || (file.fail() && !file.eof())) {
00035         entries.clear();
00036         return false;
00037     }
00038     return true;
00039 }
00040
00041 bool Index::find(int zipCode, std::streamoff& position) const {
00042     auto entry = entries.find(zipCode);
00043     if (entry == entries.end()) {
00044         return false;
00045     }
00046     position = entry->second;
00047     return true;
00048 }
```

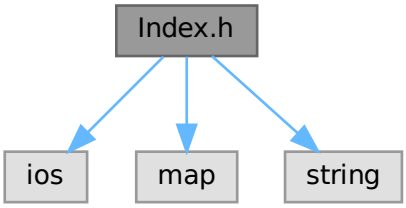
4.5 Index.h File Reference

Maps ZIP codes to record positions in a CSV file.

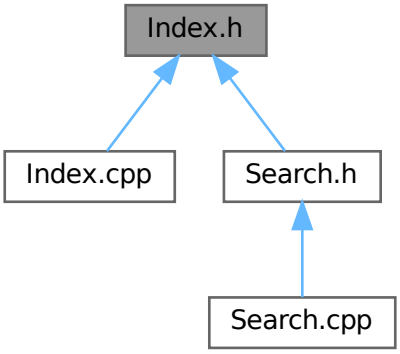
```
#include <ios>
#include <map>
```

```
#include <string>
```

Include dependency graph for Index.h:



This graph shows which files directly or indirectly include this file:



Classes

- class Index

4.5.1 Detailed Description

Maps ZIP codes to record positions in a CSV file.

Definition in file Index.h.

4.6 Index.h

Go to the documentation of this file.

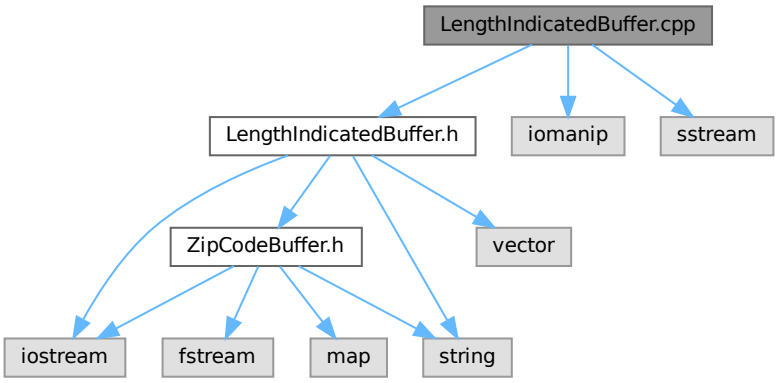
```
00001 /** @file Index.h
00002  * @brief Maps ZIP codes to record positions in a CSV file.
00003  */
00004 #ifndef INDEX_H
00005 #define INDEX_H
00006
00007 #include <ios>
00008 #include <map>
00009 #include <string>
00010
00011 class Index {
00012 public:
00013     /** @brief Builds an index for a CSV with ZIP code in its first column.
00014      * @return false on an open or read error; the index is then empty.
00015      * @details Skips the header and invalid ZIP fields. For duplicate ZIP
00016      * codes, keeps the first record. Rebuild after changing the CSV file.
00017      */
00018     bool build(const std::string& fileName);
00019
00020     /** @brief Finds a ZIP's byte offset without changing position on failure. */
00021     bool find(int zipCode, std::streamoff& position) const;
00022
00023 private:
00024     std::map<int, std::streamoff> entries;
00025 };
00026
00027 #endif
```

4.7 LengthIndicatedBuffer.cpp File Reference

Implements the length indicated buffer.

```
#include "LengthIndicatedBuffer.h"
#include <iomanip>
#include <sstream>
```

Include dependency graph for LengthIndicatedBuffer.cpp:



4.7.1 Detailed Description

Implements the length indicated buffer.

Definition in file LengthIndicatedBuffer.cpp.

4.8 LengthIndicatedBuffer.cpp

Go to the documentation of this file.

```
00001 /** @file LengthIndicatedBuffer.cpp
00002  * @brief Implements the length indicated buffer.
00003  */
00004 #include "LengthIndicatedBuffer.h"
00005 #include <iomanip>
00006 #include <sstream>
00007
00008
00009 LengthIndicatedBuffer::LengthIndicatedBuffer(){
00010     record = "";
00011 }
00012
00013 bool LengthIndicatedBuffer::read(istream& file){
00014     // The first 4 characters are the length
00015     char lengthText[5] = "";
00016     if(!file.read(lengthText, 4)){
00017         return false;
00018     }
00019
00020     int length = atoi(lengthText);
00021     if(length <= 0){
00022         cout « "Error: bad record length " « lengthText « endl;
00023         return false;
00024     }
00025
00026     // Read that many characters for the record
00027     record = string(length, ' ');
00028     if(!file.read(&record[0], length)){
00029         cout « "Error: record is cut off" « endl;
00030         return false;
00031     }
00032
00033     // skip the newline at the end (\r\n on Windows)
00034     if(file.peek() == '\r'){
00035         file.get();
00036     }
00037     if(file.peek() == '\n'){
00038         file.get();
00039     }
00040
00041     return true;
00042 }
00043
00044 bool LengthIndicatedBuffer::unpack(Buffer& buffer, vector<string> fieldNames){
00045     vector<string> fields = getFields();
00046
00047     string zip;
00048     string place;
00049     string state;
00050     string lat;
00051     string lon;
00052
00053     // Use the field names so it still works if the columns are in a different order
00054     for(int i = 0; i < fields.size() && i < fieldNames.size(); i++){
00055         if(fieldNames[i] == "Zip Code"){
00056             zip = fields[i];
00057         }
00058         else if(fieldNames[i] == "Place Name"){
00059             place = fields[i];
00060         }
00061         else if(fieldNames[i] == "State"){
00062             state = fields[i];
00063         }
```

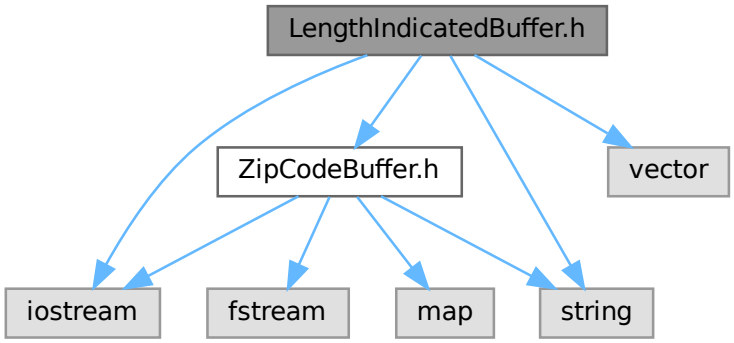
```
00064         else if(fieldNames[i] == "Lat"){
00065             lat = fields[i];
00066         }
00067         else if(fieldNames[i] == "Long"){
00068             lon = fields[i];
00069         }
00070     }
00071
00072     // skip any record that is missing data
00073     if(zip == "" || state == "" || lat == "" || lon == ""){
00074         return false;
00075     }
00076
00077     buffer.setZipCode(stoi(zip));
00078     buffer.setPlaceName(place);
00079     buffer.setState(state);
00080     buffer.setLatitude(stod(lat));
00081     buffer.setLongitude(stod(lon));
00082
00083     return true;
00084 }
00085
00086 void LengthIndicatedBuffer::pack(string line){
00087     // take off the \r so it isn't counted in the length
00088     if(line != "" && line.back() == '\r'){
00089         line.pop_back();
00090     }
00091     record = line;
00092 }
00093
00094 void LengthIndicatedBuffer::write(ostream& file){
00095     // setw and setfill('0') make the length 4 digits, like 0042
00096     file << setw(4) << setfill('0') << record.length() << record << endl;
00097 }
00098
00099 string LengthIndicatedBuffer::getRecord(){
00100     return record;
00101 }
00102
00103 vector<string> LengthIndicatedBuffer::getFields(){
00104     vector<string> fields;
00105     stringstream ss(record);
00106     string field;
00107
00108     while(getline(ss, field, ',')){
00109         fields.push_back(field);
00110     }
00111
00112     return fields;
00113 }
```

4.9 LengthIndicatedBuffer.h File Reference

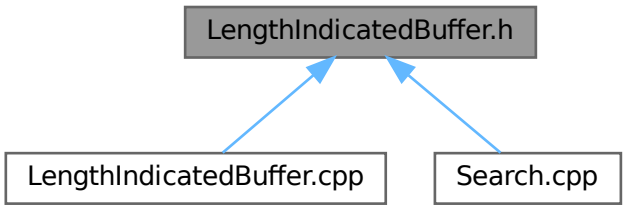
Buffer for length indicated records.

```
#include "ZipCodeBuffer.h"
#include <iostream>
#include <string>
#include <vector>
```

Include dependency graph for LengthIndicatedBuffer.h:



This graph shows which files directly or indirectly include this file:



Classes

- class LengthIndicatedBuffer

  Reads and writes one length indicated record at a time.

4.9.1 Detailed Description

Buffer for length indicated records.

Definition in file LengthIndicatedBuffer.h.

4.10 LengthIndicatedBuffer.h

Go to the documentation of this file.

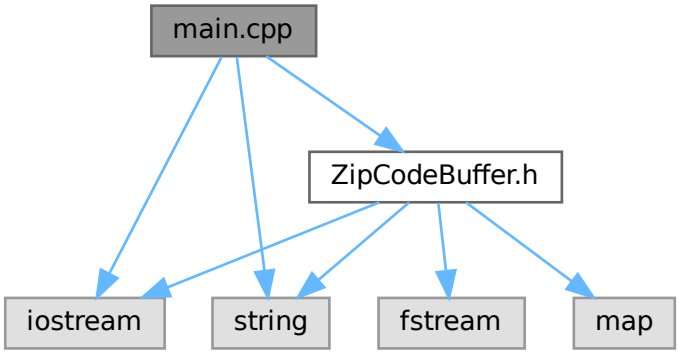
```
00001 /** @file LengthIndicatedBuffer.h
00002  * @brief Buffer for length indicated records.
00003  */
00004 #ifndef LENGTHINDICATEDBUFFER_H
00005 #define LENGTHINDICATEDBUFFER_H
00006 #include "ZipCodeBuffer.h"
00007 #include <iostream>
00008 #include <string>
00009 #include <vector>
00010 using namespace std;
00011
00012 /** @brief Reads and writes one length indicated record at a time. */
00013 class LengthIndicatedBuffer{
00014 public:
00015
00016     /** @brief Constructor. */
00017     LengthIndicatedBuffer();
00018
00019     /** @brief Reads the next record.
00020      * @param file Input file.
00021      * @return false at end of file or bad record.
00022      */
00023     bool read(istream& file);
00024
00025     /** @brief Unpacks the record into a Buffer.
00026      * @param buffer Buffer to fill.
00027      * @param fieldNames Field names in order.
00028      * @return false if the record is missing data.
00029      */
00030     bool unpack(Buffer& buffer, vector<string> fieldNames);
00031
00032     /** @brief Stores a record.
00033      * @param line Comma separated fields.
00034      */
00035     void pack(string line);
00036
00037     /** @brief Writes the record with its length.
00038      * @param file Output file.
00039      */
00040     void write(ostream& file);
00041
00042     /** @brief Returns the record. */
00043     string getRecord();
00044
00045     /** @brief Returns the record's fields. */
00046     vector<string> getFields();
00047
00048 private:
00049     string record; // Current record.
00050 };
00051
00052 #endif
```

4.11 main.cpp File Reference

Application entry point for the ZIP Code group project.

```
#include <iostream>
#include <string>
#include "ZipCodeBuffer.h"
```

Include dependency graph for main.cpp:



Functions

- int main ()

  Runs the ZIP code table application.

4.11.1 Detailed Description

Application entry point for the ZIP Code group project.

Reads ZIP code data from two CSV files and displays the results using the Buffer class.

Definition in file main.cpp.

4.11.2 Function Documentation

4.11.2.1 main()

```
int main ( )
```

Runs the ZIP code table application.

Reads us_postal_codes.csv and us_postal_codes_ROWS_RANDOMIZED.csv from the working directory, processes ZIP code records, and displays the resulting table.

Returns

0 on success; 1 if a CSV file cannot be read.

Definition at line 26 of file main.cpp.

```
00026            {
00027
00028     // CSV file names
00029     string fileName = "us_postal_codes.csv";
00030     string randomFileName =
00031         "us_postal_codes_ROWS_RANDOMIZED.csv";
00032
00033     // Create the ZIP code buffer
00034     Buffer buffer;
00035
00036     // Read the original CSV file
00037     if (!buffer.readCSV(fileName)) {
00038         cerr « "Error: could not open or read "
00039              « fileName « endl;
00040         return 1;
00041     }
00042
00043     // Read the randomized CSV file
00044     if (!buffer.readCSV(randomFileName)) {
00045         cerr « "Error: could not open or read "
00046              « randomFileName « endl;
00047         return 1;
00048     }
00049
00050     // Display the ZIP code table
00051     cout « "\nZIP Code Results\n";
00052     cout « "----------------------------\n";
00053
00054     buffer.displayTable();
00055
00056     cout « "\nProgram completed successfully."
00057          « endl;
00058
00059     return 0;
00060 }
```

References Buffer::displayTable(), and Buffer::readCSV().

4.12 main.cpp

Go to the documentation of this file.

```
00001
00002 /**
00003  * @file main.cpp
00004  * @brief Application entry point for the ZIP Code group project.
00005  *
00006  * Reads ZIP code data from two CSV files and displays
00007  * the results using the Buffer class.
00008  */
00009
00010 #include <iostream>
00011 #include <string>
00012 #include "ZipCodeBuffer.h"
00013
00014 using namespace std;
00015
00016 /**
00017  * @brief Runs the ZIP code table application.
00018  *
00019  * @details Reads us_postal_codes.csv and
00020  * us_postal_codes_ROWS_RANDOMIZED.csv from the
00021  * working directory, processes ZIP code records,
00022  * and displays the resulting table.
00023  *
00024  * @return 0 on success; 1 if a CSV file cannot be read.
00025  */
00026 int main() {
```

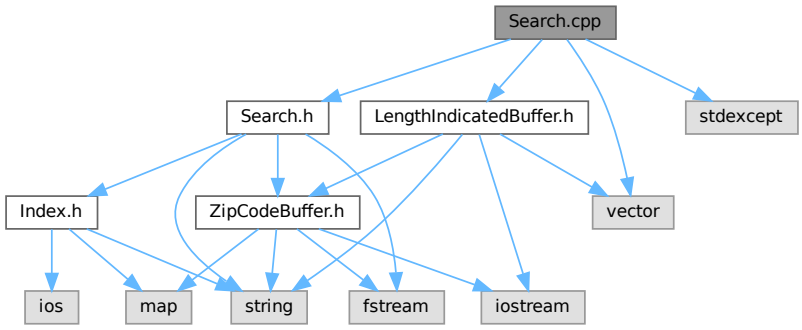
```
00027
00028     // CSV file names
00029     string fileName = "us_postal_codes.csv";
00030     string randomFileName =
00031         "us_postal_codes_ROWS_RANDOMIZED.csv";
00032
00033     // Create the ZIP code buffer
00034     Buffer buffer;
00035
00036     // Read the original CSV file
00037     if (!buffer.readCSV(fileName)) {
00038         cerr « "Error: could not open or read "
00039              « fileName « endl;
00040         return 1;
00041     }
00042
00043     // Read the randomized CSV file
00044     if (!buffer.readCSV(randomFileName)) {
00045         cerr « "Error: could not open or read "
00046              « randomFileName « endl;
00047         return 1;
00048     }
00049
00050     // Display the ZIP code table
00051     cout « "\nZIP Code Results\n";
00052     cout « "-----------------------------\n";
00053
00054     buffer.displayTable();
00055
00056     cout « "\nProgram completed successfully."
00057          « endl;
00058
00059     return 0;
00060 }
```

4.13 Search.cpp File Reference

Seeks directly to indexed ZIP records and unpacks their fields.

```
#include "Search.h"
#include "LengthIndicatedBuffer.h"
#include <stdexcept>
#include <vector>
```

Include dependency graph for Search.cpp:



4.13.1 Detailed Description

Seeks directly to indexed ZIP records and unpacks their fields.

Definition in file Search.cpp.

4.14 Search.cpp

Go to the documentation of this file.

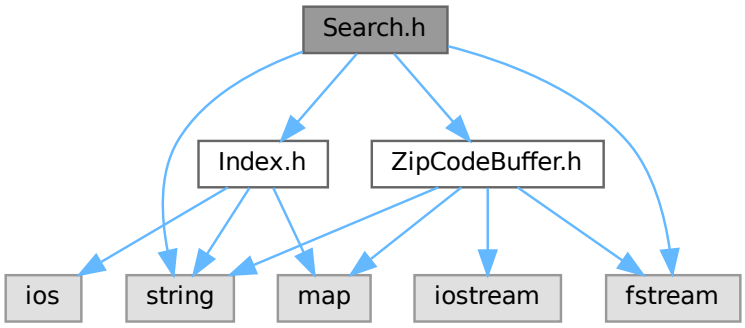
```
00001 /** @file Search.cpp
00002  * @brief Seeks directly to indexed ZIP records and unpacks their fields.
00003  */
00004 #include "Search.h"
00005 #include "LengthIndicatedBuffer.h"
00006 #include <stdexcept>
00007 #include <vector>
00008
00009 bool Search::open(const std::string& fileName) {
00010     if (file.is_open()) {
00011         file.close();
00012     }
00013     file.clear();
00014     if (!index.build(fileName)) {
00015         return false;
00016     }
00017     file.open(fileName, std::ios::binary);
00018     return file.is_open();
00019 }
00020
00021 bool Search::findZipCode(int zipCode, Buffer& buffer) {
00022     std::streamoff position;
00023     if (!file.is_open() || !index.find(zipCode, position)) {
00024         return false;
00025     }
00026     file.clear();
00027     file.seekg(position);
00028     std::string line;
00029     if (!std::getline(file, line)) {
00030         return false;
00031     }
00032
00033     LengthIndicatedBuffer record;
00034     record.pack(line);
00035     Buffer result;
00036     const std::vector<std::string> fieldNames = {
00037         "Zip Code", "Place Name", "State", "County", "Lat", "Long"
00038     };
00039     try {
00040         if (!record.unpack(result, fieldNames) || result.getZipCode() != zipCode) {
00041             return false;
00042         }
00043     } catch (const std::invalid_argument&) {
00044         return false;
00045     } catch (const std::out_of_range&) {
00046         return false;
00047     }
00048
00049     // Copy only record fields; retain the caller's accumulated extrema.
00050     buffer.setZipCode(result.getZipCode());
00051     buffer.setPlaceName(result.getPlaceName());
00052     buffer.setState(result.getState());
00053     buffer.setLatitude(result.getLatitude());
00054     buffer.setLongitude(result.getLongitude());
00055     return true;
00056 }
```

4.15 Search.h File Reference

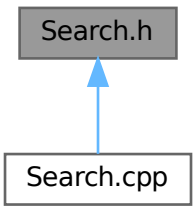
Retrieves indexed CSV records into the existing Buffer class.

```
#include "Index.h"
#include "ZipCodeBuffer.h"
#include <fstream>
#include <string>
```

Include dependency graph for Search.h:



This graph shows which files directly or indirectly include this file:



Classes

- class Search

4.15.1 Detailed Description

Retrieves indexed CSV records into the existing Buffer class.

Definition in file Search.h.

4.16 Search.h

Go to the documentation of this file.

```
00001 /** @file Search.h
00002  * @brief Retrieves indexed CSV records into the existing Buffer class.
00003  */
00004 #ifndef SEARCH_H
00005 #define SEARCH_H
00006
00007 #include "Index.h"
00008 #include "ZipCodeBuffer.h"
00009 #include <fstream>
00010 #include <string>
00011
00012 class Search {
00013 public:
00014     /** @brief Opens and indexes a CSV in the existing six-column format.
00015      * @details Supports both ordered and randomized CSV files. The file must
00016      * remain unchanged until the next open(). Does not read DAT files.
00017      * @return false if the file cannot be opened or indexed.
00018      */
00019     bool open(const std::string& fileName);
00020
00021     /** @brief Loads a ZIP record into buffer; leading zeros are optional.
00022      * @return false if unopened, absent, unreadable, or malformed.
00023      * @details Leaves buffer unchanged on failure. Uses the existing comma
00024      * splitting convention, which does not support quoted commas.
00025      */
00026     bool findZipCode(int zipCode, Buffer& buffer);
00027
00028 private:
00029     Index index;
00030     std::ifstream file;
00031 };
00032
00033 #endif
```

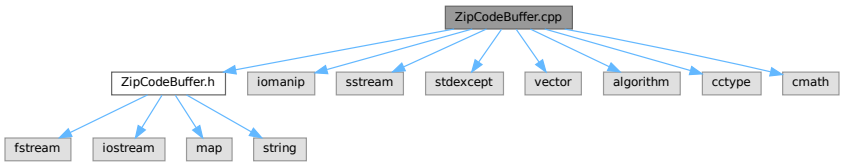
4.17 ZipCodeBuffer.cpp File Reference

Implements ZIP code CSV parsing and geographic extrema.

```
#include "ZipCodeBuffer.h"
#include <iomanip>
#include <sstream>
#include <stdexcept>
#include <vector>
#include <algorithm>
#include <cctype>
```

```
#include <cmath>
```

Include dependency graph for ZipCodeBuffer.cpp:



Functions

- static string trim (const string &value)
- static vector< string > splitCSV (const string &line)
- static int parseInteger (const string &value)
- static double parseDouble (const string &value)

4.17.1 Detailed Description

Implements ZIP code CSV parsing and geographic extrema.

Definition in file ZipCodeBuffer.cpp.

4.17.2 Function Documentation

4.17.2.1 parseDouble()

```
static double parseDouble (
            const string & value )  [static]
```

Definition at line 68 of file ZipCodeBuffer.cpp.

```
00068                                                {
00069     size_t pos = 0;
00070     double number = stod(value, &pos);
00071
00072     if (pos != value.size() || !isfinite(number)) {
00073         throw invalid_argument("Invalid coordinate: " + value);
00074     }
00075
00076     return number;
00077 }
```

Referenced by Buffer::readCSV().

4.17.2.2 parseInteger()

```
static int parseInteger (
            const string & value )  [static]
```

Definition at line 57 of file ZipCodeBuffer.cpp.

```
00057                                                {
00058     size_t pos = 0;
00059     int number = stoi(value, &pos);
00060
00061     if (pos != value.size()) {
00062         throw invalid_argument("Invalid ZIP: " + value);
00063     }
00064
00065     return number;
00066 }
```

Referenced by Buffer::readCSV().

4.17.2.3 splitCSV()

```
static vector< string > splitCSV (
            const string & line )  [static]
```

Definition at line 28 of file ZipCodeBuffer.cpp.

```
00028                                                    {
00029     vector<string> fields;
00030     string field;
00031     bool quoted = false;
00032
00033     for (size_t i = 0; i < line.size(); ++i) {
00034         char c = line[i];
00035
00036         if (c == '"') {
00037             if (quoted && i + 1 < line.size()
00038                     && line[i + 1] == '"') {
00039                 field += '"';
00040                 ++i;
00041             } else {
00042                 quoted = !quoted;
00043             }
00044         } else if (c == ',' && !quoted) {
00045             fields.push_back(trim(field));
00046             field.clear();
00047         } else {
00048             field += c;
00049         }
00050     }
00051
00052     fields.push_back(trim(field));
00053     return fields;
00054 }
```

References trim().

Referenced by Buffer::readCSV().

4.17.2.4 trim()

```
static string trim (
            const string & value )  [static]
```

Definition at line 19 of file ZipCodeBuffer.cpp.

```
00019                                            {
00020     size_t start = value.find_first_not_of(" \t\r\n");
00021     if (start == string::npos) return "";
00022
00023     size_t end = value.find_last_not_of(" \t\r\n");
00024     return value.substr(start, end - start + 1);
00025 }
```

Referenced by splitCSV().

4.18 ZipCodeBuffer.cpp

Go to the documentation of this file.

```
00001
00002 /**
00003  * @file ZipCodeBuffer.cpp
00004  * @brief Implements ZIP code CSV parsing and geographic extrema.
00005  */
00006
00007 #include "ZipCodeBuffer.h"
00008 #include <iomanip>
00009 #include <sstream>
00010 #include <stdexcept>
00011 #include <vector>
00012 #include <algorithm>
00013 #include <cctype>
00014 #include <cmath>
00015
00016 using namespace std;
00017
00018 // Remove surrounding whitespace.
00019 static string trim(const string& value) {
00020     size_t start = value.find_first_not_of(" \t\r\n");
00021     if (start == string::npos) return "";
00022
00023     size_t end = value.find_last_not_of(" \t\r\n");
00024     return value.substr(start, end - start + 1);
00025 }
00026
00027 // Split a CSV row while respecting quoted fields.
00028 static vector<string> splitCSV(const string& line) {
00029     vector<string> fields;
00030     string field;
00031     bool quoted = false;
00032
00033     for (size_t i = 0; i < line.size(); ++i) {
00034         char c = line[i];
00035
00036         if (c == '"') {
00037             if (quoted && i + 1 < line.size()
00038                     && line[i + 1] == '"') {
00039                 field += '"';
00040                 ++i;
00041             } else {
00042                 quoted = !quoted;
00043             }
00044         } else if (c == ',' && !quoted) {
00045             fields.push_back(trim(field));
00046             field.clear();
00047         } else {
00048             field += c;
00049         }
00050     }
00051
00052     fields.push_back(trim(field));
00053     return fields;
00054 }
00055
00056 // Convert numeric strings, rejecting partial conversions.
00057 static int parseInteger(const string& value) {
00058     size_t pos = 0;
00059     int number = stoi(value, &pos);
00060
00061     if (pos != value.size()) {
00062         throw invalid_argument("Invalid ZIP: " + value);
00063     }
00064
00065     return number;
00066 }
00067
00068 static double parseDouble(const string& value) {
00069     size_t pos = 0;
00070     double number = stod(value, &pos);
00071
00072     if (pos != value.size() || !isfinite(number)) {
00073         throw invalid_argument("Invalid coordinate: " + value);
00074     }
00075
00076     return number;
```

```
00077 }
00078
00079 /**
00080  * @brief Initializes an empty ZIP code record.
00081  */
00082 Buffer::Buffer() {
00083     state = "";
00084     placeName = "";
00085     latitude = 0.0;
00086     longitude = 0.0;
00087     zipCode = 0;
00088 }
00089
00090 void Buffer::setState(string state) {
00091     this->state = state;
00092 }
00093
00094 void Buffer::setPlaceName(string placeName) {
00095     this->placeName = placeName;
00096 }
00097
00098 void Buffer::setLatitude(double latitude) {
00099     this->latitude = latitude;
00100 }
00101
00102 void Buffer::setLongitude(double longitude) {
00103     this->longitude = longitude;
00104 }
00105
00106 void Buffer::setZipCode(int zipCode) {
00107     this->zipCode = zipCode;
00108 }
00109
00110 string Buffer::getState() {
00111     return state;
00112 }
00113
00114 string Buffer::getPlaceName() {
00115     return placeName;
00116 }
00117
00118 double Buffer::getLatitude() {
00119     return latitude;
00120 }
00121
00122 double Buffer::getLongitude() {
00123     return longitude;
00124 }
00125
00126 int Buffer::getZipCode() {
00127     return zipCode;
00128 }
00129
00130 /**
00131  * @brief Reads ZIP code records from a CSV file.
00132  * @param fileName CSV file path.
00133  * @return true if opened and read; false otherwise.
00134  *
00135  * @details Invalid or incomplete records are skipped.
00136  * Existing extrema are preserved across calls.
00137  */
00138 bool Buffer::readCSV(string fileName) {
00139     ifstream file(fileName);
00140
00141     if (!file.is_open()) {
00142         cerr « "Error: Cannot open " « fileName « endl;
00143         return false;
00144     }
00145
00146     string line;
00147     size_t lineNumber = 0;
00148     size_t processed = 0;
00149     size_t skipped = 0;
00150
00151     // Skip CSV header.
00152     if (!getline(file, line)) {
00153         cerr « "Error: Empty CSV: " « fileName « endl;
00154         return false;
00155     }
00156
00157     ++lineNumber;
```

```
00158
00159     while (getline(file, line)) {
00160         ++lineNumber;
00161
00162         vector<string> fields = splitCSV(line);
00163
00164         // Expected columns:
00165         // ZIP, Place, State, County, Latitude, Longitude
00166         if (fields.size() < 6) {
00167             ++skipped;
00168             continue;
00169         }
00170
00171         const string& zip = fields[0];
00172         const string& place = fields[1];
00173         const string& stateValue = fields[2];
00174         const string& lat = fields[4];
00175         const string& lon = fields[5];
00176
00177         if (zip.empty() || stateValue.empty() ||
00178             lat.empty() || lon.empty()) {
00179             ++skipped;
00180             continue;
00181         }
00182
00183         try {
00184             int parsedZip = parseInteger(zip);
00185             double parsedLat = parseDouble(lat);
00186             double parsedLon = parseDouble(lon);
00187
00188             if (parsedZip < 0 || parsedZip > 99999 ||
00189                 parsedLat < -90 || parsedLat > 90 ||
00190                 parsedLon < -180 || parsedLon > 180) {
00191                 ++skipped;
00192                 continue;
00193             }
00194
00195             setZipCode(parsedZip);
00196             setPlaceName(place);
00197             setState(stateValue);
00198             setLatitude(parsedLat);
00199             setLongitude(parsedLon);
00200
00201             findExtrema();
00202             ++processed;
00203
00204         } catch (const exception& e) {
00205             ++skipped;
00206
00207             // Limit diagnostic output for large datasets.
00208             if (skipped <= 10) {
00209                 cerr « "Skipping invalid record at line "
00210                      « lineNumber « " in " « fileName
00211                      « ": " « e.what() « endl;
00212             }
00213         }
00214     }
00215
00216     if (file.bad()) {
00217         cerr « "Error reading file: " « fileName « endl;
00218         return false;
00219     }
00220
00221     cout « "File: " « fileName « endl;
00222     cout « "Processed records: " « processed « endl;
00223     cout « "Skipped records: " « skipped « endl;
00224
00225     if (processed == 0) {
00226         cerr « "Error: No valid records in "
00227              « fileName « endl;
00228         return false;
00229     }
00230
00231     return true;
00232 }
00233
00234 /**
00235  * @brief Finds easternmost, westernmost, northernmost,
00236  * and southernmost ZIP codes for each state.
00237  *
00238  * @details Coordinate ties select the smaller ZIP code.
```

```
00239  */
00240 void Buffer::findExtrema() {
00241
00242     if (extremes.count(state) == 0) {
00243         Extremes extreme;
00244
00245         extreme.east = zipCode;
00246         extreme.west = zipCode;
00247         extreme.north = zipCode;
00248         extreme.south = zipCode;
00249
00250         extreme.eastLongitude = longitude;
00251         extreme.westLongitude = longitude;
00252         extreme.northLatitude = latitude;
00253         extreme.southLatitude = latitude;
00254
00255         extremes[state] = extreme;
00256         return;
00257     }
00258
00259     Extremes& extreme = extremes[state];
00260
00261     // Easternmost: highest longitude.
00262     if (longitude > extreme.eastLongitude ||
00263         (longitude == extreme.eastLongitude &&
00264          zipCode < extreme.east)) {
00265         extreme.east = zipCode;
00266         extreme.eastLongitude = longitude;
00267     }
00268
00269     // Westernmost: lowest longitude.
00270     if (longitude < extreme.westLongitude ||
00271         (longitude == extreme.westLongitude &&
00272          zipCode < extreme.west)) {
00273         extreme.west = zipCode;
00274         extreme.westLongitude = longitude;
00275     }
00276
00277     // Northernmost: highest latitude.
00278     if (latitude > extreme.northLatitude ||
00279         (latitude == extreme.northLatitude &&
00280          zipCode < extreme.north)) {
00281         extreme.north = zipCode;
00282         extreme.northLatitude = latitude;
00283     }
00284
00285     // Southernmost: lowest latitude.
00286     if (latitude < extreme.southLatitude ||
00287         (latitude == extreme.southLatitude &&
00288          zipCode < extreme.south)) {
00289         extreme.south = zipCode;
00290         extreme.southLatitude = latitude;
00291     }
00292 }
00293
00294 /**
00295  * @brief Returns the easternmost ZIP code.
00296  */
00297 int Buffer::getEasternmostZipCode(string state) {
00298     return extremes.at(state).east;
00299 }
00300
00301 /**
00302  * @brief Returns the westernmost ZIP code.
00303  */
00304 int Buffer::getWesternmostZipCode(string state) {
00305     return extremes.at(state).west;
00306 }
00307
00308 /**
00309  * @brief Returns the northernmost ZIP code.
00310  */
00311 int Buffer::getNorthernmostZipCode(string state) {
00312     return extremes.at(state).north;
00313 }
00314
00315 /**
00316  * @brief Returns the southernmost ZIP code.
00317  */
00318 int Buffer::getSouthernmostZipCode(string state) {
00319     return extremes.at(state).south;
```

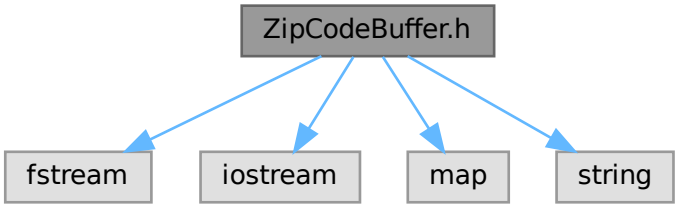
```
00320 }
00321
00322 /**
00323  * @brief Displays geographic extrema sorted by state.
00324  */
00325 void Buffer::displayTable() {
00326
00327     cout « left « setfill(' ');
00328     cout « setw(7) « "State"
00329          « setw(14) « "Easternmost"
00330          « setw(14) « "Westernmost"
00331          « setw(14) « "Northernmost"
00332          « setw(14) « "Southernmost"
00333          « endl;
00334
00335     cout « string(63, '-') « endl;
00336
00337     for (const auto& entry : extremes) {
00338         const string& stateValue = entry.first;
00339
00340         cout « left « setfill(' ')
00341              « setw(7) « stateValue;
00342
00343         cout « right « setfill('0');
00344
00345         cout « setw(5)
00346              « getEasternmostZipCode(stateValue)
00347              « "          ";
00348
00349         cout « setw(5)
00350              « getWesternmostZipCode(stateValue)
00351              « "          ";
00352
00353         cout « setw(5)
00354              « getNorthernmostZipCode(stateValue)
00355              « "          ";
00356
00357         cout « setw(5)
00358              « getSouthernmostZipCode(stateValue)
00359              « endl;
00360     }
00361
00362     cout « setfill(' ') « left;
00363 }
```

4.19 ZipCodeBuffer.h File Reference

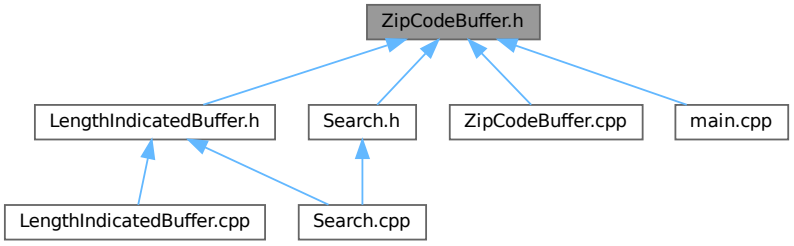
Record buffer and per-state geographic extrema declarations.

```
#include <fstream>
#include <iostream>
#include <map>
#include <string>
```

Include dependency graph for ZipCodeBuffer.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct Extremes

  ZIP codes and comparison coordinates for one state.
- class Buffer

  Reads records sequentially and accumulates geographic extrema.

4.19.1 Detailed Description

Record buffer and per-state geographic extrema declarations.

Definition in file ZipCodeBuffer.h.

4.20 ZipCodeBuffer.h

Go to the documentation of this file.

```
00001 /** @file ZipCodeBuffer.h
00002  * @brief Record buffer and per-state geographic extrema declarations.
00003  */
00004 #ifndef ZIPCODEBUFFER_H
00005 #define ZIPCODEBUFFER_H
00006 #include <fstream>
00007 #include <iostream>
00008 #include <map>
00009 #include <string>
00010 using namespace std;
00011
00012 /** @brief ZIP codes and comparison coordinates for one state. */
00013 struct Extremes{
00014     int east; ///< Stored east value.
00015     int west; ///< Stored west value.
00016     int north; ///< Stored north value.
00017     int south; ///< Stored south value.
00018
00019     double eastLongitude; ///< Stored eastLongitude value.
00020     double westLongitude; ///< Stored westLongitude value.
00021     double northLatitude; ///< Stored northLatitude value.
00022     double southLatitude; ///< Stored southLatitude value.
00023 };
00024
00025 /** @brief Reads records sequentially and accumulates geographic extrema.
00026  * @details Stores the current record and a state-keyed map. Equal coordinates
00027  * select the smaller ZIP code, making results independent of record order.
00028  */
00029 class Buffer{
00030 public:
00031
00032     /** @brief Initializes an empty record and empty extrema map. */
00033     Buffer();
00034
00035     /** @brief Sets the current record State.
00036      * @param state Value to store in the record buffer.
00037      */
00038     void setState(string state);
00039
00040     /** @brief Sets the current record PlaceName.
00041      * @param placeName Value to store in the record buffer.
00042      */
00043     void setPlaceName(string placeName);
00044
00045     /** @brief Sets the current record Latitude.
00046      * @param latitude Value to store in the record buffer.
00047      */
00048     void setLatitude(double latitude);
00049
00050     /** @brief Sets the current record Longitude.
00051      * @param longitude Value to store in the record buffer.
00052      */
00053     void setLongitude(double longitude);
00054
00055     /** @brief Sets the current record ZipCode.
00056      * @param zipCode Value to store in the record buffer.
00057      */
00058     void setZipCode(int zipCode);
00059
00060     /** @brief Returns State.
00061      * @return Stored value.
00062      */
00063     string getState();
00064
00065     /** @brief Returns PlaceName.
00066      * @return Stored value.
00067      */
00068     string getPlaceName();
00069
00070     /** @brief Returns Latitude.
00071      * @return Stored value.
00072      */
00073     double getLatitude();
00074
00075     /** @brief Returns Longitude.
00076      * @return Stored value.
```

```
00077      */
00078     double getLongitude();
00079
00080     /** @brief Returns ZipCode.
00081      * @return Stored value.
00082      */
00083     int getZipCode();
00084
00085     /** @brief Reads the CSV and accumulates one record at a time.
00086      * @param fileName Path to a CSV with a header and columns ZIP, place, state, county, latitude,
      longitude.
00087      * @return false if opening fails; true after processing the file.
00088      * @details Skips records with empty ZIP, state, latitude, or longitude.
00089      * Existing extrema remain when called again. Comma splitting does not handle quoted commas.
00090      * @throws std::invalid_argument If a numeric field cannot be converted.
00091      * @throws std::out_of_range If a numeric field is outside the conversion range.
00092      */
00093     bool readCSV(string fileName);
00094
00095     /** @brief Updates state extrema using the current buffered record.
00096      * @details East uses maximum longitude, west minimum longitude, north
00097      * maximum latitude, and south minimum latitude. Ties use the smaller ZIP.
00098      */
00099     void findExtrema();
00100
00101     /** @brief Returns EasternmostZipCode.
00102      * @param state State abbreviation already present in the extrema map.
00103      * @pre The state has been accumulated by findExtrema().
00104      * @return ZIP code for the requested direction.
00105      * @note An unknown state inserts a zero-initialized map entry.
00106      */
00107     int getEasternmostZipCode(string state);
00108
00109     /** @brief Returns WesternmostZipCode.
00110      * @param state State abbreviation already present in the extrema map.
00111      * @pre The state has been accumulated by findExtrema().
00112      * @return ZIP code for the requested direction.
00113      * @note An unknown state inserts a zero-initialized map entry.
00114      */
00115     int getWesternmostZipCode(string state);
00116
00117     /** @brief Returns NorthernmostZipCode.
00118      * @param state State abbreviation already present in the extrema map.
00119      * @pre The state has been accumulated by findExtrema().
00120      * @return ZIP code for the requested direction.
00121      * @note An unknown state inserts a zero-initialized map entry.
00122      */
00123     int getNorthernmostZipCode(string state);
00124
00125     /** @brief Returns SouthernmostZipCode.
00126      * @param state State abbreviation already present in the extrema map.
00127      * @pre The state has been accumulated by findExtrema().
00128      * @return ZIP code for the requested direction.
00129      * @note An unknown state inserts a zero-initialized map entry.
00130      */
00131     int getSouthernmostZipCode(string state);
00132
00133     /** @brief Prints the state extrema table to standard output.
00134      * @details Rows follow alphabetical state order. Columns are state, east,
00135      * west, north, south. ZIP codes are padded to five digits.
00136      */
00137     void displayTable();
00138
00139 private:
00140
00141     string state; ///< Current record state.
00142     string placeName; ///< Current record placeName.
00143     double latitude; ///< Current record latitude.
00144     double longitude; ///< Current record longitude.
00145     int zipCode; ///< Current record zipCode.
00146
00147     map<string, Extremes> extremes; // one entry per state, a map keeps them in alphabetical order
00148
00149 };
00150
00151 #endif
```

Index

analyzeCSV
    CSVConversion.cpp, 36

Buffer, 5
    Buffer, 6
    displayTable, 7
    extremes, 17
    findExtrema, 7
    getEasternmostZipCode, 8
    getLatitude, 9
    getLongitude, 9
    getNorthernmostZipCode, 10
    getPlaceName, 10
    getSouthernmostZipCode, 11
    getState, 11
    getWesternmostZipCode, 12
    getZipCode, 12
    latitude, 17
    longitude, 17
    placeName, 18
    readCSV, 13
    setLatitude, 15
    setLongitude, 15
    setPlaceName, 16
    setState, 16
    setZipCode, 17
    state, 18
    zipCode, 18
build
    Index, 25

convertCSVTodat
    CSVConversion.cpp, 36
CSVConversion.cpp, 35
    analyzeCSV, 36
    convertCSVTodat, 36
    main, 37
    writeHeaderToFile, 37

displayTable
    Buffer, 7

east
    Extremes, 19
eastLongitude
    Extremes, 19

entries
    Index, 26
Extremes, 19
    east, 19
    eastLongitude, 19
    north, 20
    northLatitude, 20
    south, 20
    southLatitude, 20
    west, 20
    westLongitude, 21
extremes
    Buffer, 17

fieldCount
    FileHeader, 22
Fieldinfo, 21
    fieldName, 21
    fieldType, 21
fieldName
    Fieldinfo, 21
fields
    FileHeader, 22
fieldType
    Fieldinfo, 21
file
    Search, 33
FileHeader, 22
    fieldCount, 22
    fields, 22
    headerSize, 22
    primaryKeyName, 23
    recordCount, 23
    sizeFieldBytes, 23
    sizeincluded, 23
    structureType, 23
    version, 23
find
    Index, 25
findExtrema
    Buffer, 7
findZipCode
    Search, 32

getEasternmostZipCode
    Buffer, 8

getFields
LengthIndicatedBuffer, 27
getLatitude
Buffer, 9
getLongitude
Buffer, 9
getNorthernmostZipCode
Buffer, 10
getPlaceName
Buffer, 10
getRecord
LengthIndicatedBuffer, 27
getSouthernmostZipCode
Buffer, 11
getState
Buffer, 11
getWesternmostZipCode
Buffer, 12
getZipCode
Buffer, 12

headerSize
FileHeader, 22

Index, 24
build, 25
entries, 26
find, 25
index
Search, 33
Index.cpp, 39
Index.h, 40

latitude
Buffer, 17
LengthIndicatedBuffer, 26
getFields, 27
getRecord, 27
LengthIndicatedBuffer, 27
pack, 28
read, 28
record, 31
unpack, 29
write, 30
LengthIndicatedBuffer.cpp, 42
LengthIndicatedBuffer.h, 44
longitude
Buffer, 17

main
CSVConversion.cpp, 37
main.cpp, 47
main.cpp, 46
main, 47

north
Extremes, 20
northLatitude
Extremes, 20

open
Search, 32

pack
LengthIndicatedBuffer, 28
parseDouble
ZipCodeBuffer.cpp, 53
parseInteger
ZipCodeBuffer.cpp, 53
placeName
Buffer, 18
primaryKeyName
FileHeader, 23

read
LengthIndicatedBuffer, 28
readCSV
Buffer, 13
record
LengthIndicatedBuffer, 31
recordCount
FileHeader, 23

Search, 31
file, 33
findZipCode, 32
index, 33
open, 32
Search.cpp, 49
Search.h, 51
setLatitude
Buffer, 15
setLongitude
Buffer, 15
setPlaceName
Buffer, 16
setState
Buffer, 16
setZipCode
Buffer, 17
sizeFieldBytes
FileHeader, 23
sizeincluded
FileHeader, 23
south
Extremes, 20
southLatitude
Extremes, 20
splitCSV
ZipCodeBuffer.cpp, 54

state
Buffer, 18
structureType
FileHeader, 23

trim
ZipCodeBuffer.cpp, 54

unpack
LengthIndicatedBuffer, 29

version
FileHeader, 23

west
Extremes, 20
westLongitude
Extremes, 21
write
LengthIndicatedBuffer, 30
writeHeaderToFile
CSVConversion.cpp, 37

zipCode
Buffer, 18
ZipCodeBuffer.cpp, 52
parseDouble, 53
parseInteger, 53
splitCSV, 54
trim, 54
ZipCodeBuffer.h, 59